

Academic DBMS Project Viva Guide: Inventory Management System

Database Engine: MySQL Community Server (InnoDB Storage Engine) **Schema:** 8 Normalized Relational Tables (inventory_management) **Target Audience:** B.Tech CSE / IT Students preparing for DBMS Project Viva & Lab Examinations **Scope:** Relational Data Modeling, Integrity Constraints, Normalization (1NF–3NF), Keys, Joins, Aggregations, Subqueries, Analytical Views, and ACID Transactions.

Table of Contents

- [1. Tables and Relationships](#)
- [2. Database Constraints](#)
- [3. SQL Aggregate Functions](#)
- [4. SQL Concepts Used in the Project](#)
- [5. Database Normalization \(1NF, 2NF, 3NF\)](#)
- [6. Database Keys](#)
- [7. Referential Integrity](#)
- [8. Database Views](#)
- [9. Transactions and ACID Properties](#)
- [10. Important DBMS Viva Questions & Answers](#)
- [11. TOP 25 DBMS Viva Points to Memorize](#)

1. Tables and Relationships

The database consists of **8 relational tables** designed in Third Normal Form (3NF) using the **InnoDB** storage engine to support ACID transactions and foreign key referential integrity.

```
[Category] (1) —< belongs to >— (N) [Product]
                                   | (1)
                                   |—< sold in >— (N) [Sale_Details]
                                   |—< ordered in >— (N) [Purchase_Details]

[Customer] (1) —< places >— (N) [Sale] (1) —< has >— (N) [Sale_Details]

[Supplier] (1) —< supplies >— (N) [Purchase] (1) —< has >— (N) [Purchase_Details]
```

Detailed Breakdown of Every Relationship

1. Category → Product

- **Parent Table:** Category | **Child Table:** Product
- **Primary Key:** Category.Category_ID | **Foreign Key:** Product.Category_ID
- **Relationship Type:** One-to-Many (1:N)
- **Real-World Meaning:** One product category (e.g., "Storage Devices") groups many distinct products (e.g., SSD, HDD, Flash Drive). Each product belongs to exactly one category.
- **Why Required:** Eliminates category name duplication across thousands of product records.

2. Supplier → Purchase

- **Parent Table:** Supplier | **Child Table:** Purchase
- **Primary Key:** Supplier.Supplier_ID | **Foreign Key:** Purchase.Supplier_ID
- **Relationship Type:** One-to-Many (1:N)
- **Real-World Meaning:** One vendor/supplier can supply multiple purchase orders over time. Each purchase order header is billed to exactly one supplier.
- **Why Required:** Maintains vendor accountability and procurement history without repeating vendor addresses on every invoice.

3. Purchase → Purchase_Details

- **Parent Table:** Purchase | **Child Table:** Purchase_Details
- **Primary Key:** Purchase.Purchase_ID | **Foreign Key:** Purchase_Details.Purchase_ID
- **Relationship Type:** One-to-Many (1:N)
- **Real-World Meaning:** One purchase invoice header contains multiple line items (individual items bought in that order).
- **Why Required:** A purchase order can contain multiple distinct items with varying quantities and prices.

4. Product → Purchase_Details

- **Parent Table:** Product | **Child Table:** Purchase_Details
- **Primary Key:** Product.Product_ID | **Foreign Key:** Purchase_Details.Product_ID
- **Relationship Type:** One-to-Many (1:N)
- **Real-World Meaning:** A single product from our catalog can appear across multiple purchase order line items over time.
- **Why Required:** Links incoming inventory batches directly to catalog items being restocked.

5. Customer → Sale

- **Parent Table:** Customer | **Child Table:** Sale
- **Primary Key:** Customer.Customer_ID | **Foreign Key:** Sale.Customer_ID
- **Relationship Type:** One-to-Many (1:N)
- **Real-World Meaning:** One customer can place many sales orders. Each sale invoice belongs to exactly one registered customer.
- **Why Required:** Tracks customer order history and billing without storing duplicate customer profiles.

6. Sale → Sale_Details

- **Parent Table:** Sale | **Child Table:** Sale_Details
- **Primary Key:** Sale.Sale_ID | **Foreign Key:** Sale_Details.Sale_ID
- **Relationship Type:** One-to-Many (1:N)
- **Real-World Meaning:** One sales invoice contains multiple line items (individual products sold).
- **Why Required:** Allows a customer to purchase several different items in a single checkout transaction.

7. Product → Sale_Details

- **Parent Table:** Product | **Child Table:** Sale_Details
- **Primary Key:** Product.Product_ID | **Foreign Key:** Sale_Details.Product_ID
- **Relationship Type:** One-to-Many (1:N)
- **Real-World Meaning:** A product can be sold across many different customer invoices.
- **Why Required:** Links sold line items back to the product to decrement stock and track units sold.

Cardinality Types & Many-to-Many Resolution

Cardinality	Presence in Project	Explanation
One-to-One (1:1)	<i>Not explicitly used</i>	In this schema, entities with a 1:1 relationship are merged into a single table to avoid unnecessary joins.
One-to-Many (1:N)	Used in all 7 relationships	One parent row maps to zero, one, or multiple child rows.
Many-to-Many (M:N)	Resolved via associative tables	A Purchase has many Products, and a Product appears in many Purchases. A Sale has many Products, and a Product appears in many Sales.

How Detail Tables Resolve M:N Relationships:

Direct M:N relationships cannot be implemented cleanly in a relational database because storing multi-valued attributes in a single cell violates the First Normal Form (1NF).

1. **Purchase** ↔ **Product** (M:N) is resolved using the bridge/junction table **Purchase_Details** :

$$\text{\textit{Purchase}} \{ (1) \leftrightarrow (N) \text{\textit{Purchase_Details}} \} (N) \leftrightarrow (1) \text{\textit{Product}} \}$$

1. **Sale** ↔ **Product** (M:N) is resolved using the bridge/junction table **Sale_Details** :

$$\text{\textit{Sale}} \{ (1) \leftrightarrow (N) \text{\textit{Sale_Details}} \} (N) \leftrightarrow (1) \text{\textit{Product}} \}$$

Each detail record stores the **intersection data** specific to that transaction instance: **Quantity** and **Unit_Price** .

2. Database Constraints

Constraints enforce **Domain Integrity**, **Entity Integrity**, and **Referential Integrity** directly inside the database engine.

Constraint Type	Project Definition & Location	DBMS Purpose	Violation Behavior
PRIMARY KEY	pk_category on Category(Category_ID) pk_product on Product(Product_ID) (All 8 tables have auto-increment PKs)	Enforces Entity Integrity . Guarantees uniqueness and non-nullability for every row.	Rejects insert with error: Duplicate entry for key 'PRIMARY'
FOREIGN KEY	7 FK constraints defined in 03_constraints.sql (e.g., fk_product_category, fk_sale_customer)	Enforces Referential Integrity . Ensures a child row cannot reference a nonexistent parent ID.	Rejects insert/update with error: Cannot add or update a child row: a foreign key constraint fails
NOT NULL	Product_Name VARCHAR(150) NOT NULL Price DECIMAL(10,2) NOT NULL Category_Name VARCHAR(100) NOT NULL	Disallows missing or unknown data in essential attributes.	Rejects insert/update with error: Column cannot be null
UNIQUE	uq_category_name on Category(Category_Name)	Ensures no duplicate category names exist in the catalog.	Rejects insert/update with error: Duplicate entry for key 'uq_category_name'
DEFAULT	Stock_Quantity INT DEFAULT 0 Reorder_Level INT DEFAULT 10 Purchase_Date DATETIME DEFAULT CURRENT_TIMESTAMP	Provides automatic fallback values when an insert omits that attribute.	N/A (fills missing column with system default)
CHECK	chk_product_price CHECK (Price >= 0) chk_product_stock CHECK (Stock_Quantity >= 0) chk_sale_detail_qty CHECK (Quantity > 0) chk_purchase_detail_price CHECK (Unit_Price >= 0)	Enforces Domain Integrity and business logic at the storage level.	Rejects insert/update with error: Check constraint 'chk_product_price' is violated
ON DELETE RESTRICT	Configured on parent entities: Category, Supplier, Customer, Product	Prevents accidental deletion of a parent record if children depend on it.	If you try DELETE FROM Category WHERE Category_ID = 1, MySQL blocks it because products exist under it.
ON DELETE CASCADE	Configured on line items: Purchase_Details -> Purchase Sale_Details -> Sale	Guarantees automatic cleanup of dependent line items if a parent invoice header is deleted.	Deleting a Sale row automatically deletes all corresponding Sale_Details rows.
ON UPDATE CASCADE	Configured on all 7 Foreign Keys	Propagates primary key changes from parent to child rows automatically.	If a Customer_ID changes, all referencing Sale.Customer_ID values update automatically.

Common Viva Comparisons

1. Primary Key vs. Foreign Key

- **Primary Key:** Uniquely identifies a record within its **own** table. It cannot accept **NULL** values. Only one primary key is allowed per table.
- **Foreign Key:** References the Primary Key of **another** table to establish a link. It can accept duplicate values and can be **NULL** (unless marked **NOT NULL**). A table can have multiple foreign keys.

2. Primary Key vs. UNIQUE Constraint

- **Primary Key:** A table can have **only one** Primary Key. It strictly prohibits `NULL` values.
- **UNIQUE Constraint:** A table can have **multiple** UNIQUE constraints (e.g., `Category_Name`, `Email`). It allows `NULL` values (in SQL standards, multiple `NULL` s are permitted because `NULL != NULL`).

3. NOT NULL vs. UNIQUE

- **NOT NULL:** Ensures a value is provided; it does **not** check for duplicates.
- **UNIQUE:** Ensures all entered values are distinct; it does **not** prohibit `NULL` unless paired with `NOT NULL`.

4. ON DELETE RESTRICT vs. ON DELETE CASCADE

- **RESTRICT:** Rejects and rolls back the deletion of a parent row if matching child records exist (e.g., cannot delete a `Customer` who has historical `Sale` records).
- **CASCADE:** Allows deletion of the parent row and **automatically deletes** all associated child rows (e.g., deleting a `Sale` header automatically purges its `Sale_Details` line items).

5. CHECK vs. NOT NULL

- **NOT NULL:** Only checks whether the attribute contains a value versus `NULL`.
- **CHECK:** Evaluates a boolean logical expression against the attribute value (e.g., `Price >= 0`, `Quantity > 0`).

3. SQL Aggregate Functions

Aggregate functions compute a single summary value from a set of column values across multiple rows.

Project Implementations

1. COUNT()

- **Definition:** Returns the total number of rows matching the query criteria.
- **Project Example:**

```
SELECT COUNT(*) AS Total_Products FROM Product;
```

- **Result / Output:** Returns `14` (total products in inventory).

2. SUM()

- **Definition:** Calculates the mathematical sum of all numeric values in a column.
- **Project Example:**

```
SELECT SUM(Stock_Quantity * Price) AS Total_Stock_Valuation FROM Product;
```

- **Result / Output:** Returns `385030.00` (the cumulative monetary value of all stock in the warehouse).

3. AVG()

- **Definition:** Calculates the arithmetic mean of all numeric values in a column.
- **Project Example:**

```
SELECT AVG(Price) AS Average_Product_Price FROM Product;
```

- **Result / Output:** Returns the average unit price across all catalog products.

4. MIN()

- **Definition:** Identifies the minimum (smallest) value in a column.
- **Project Example:**

```
SELECT MIN(Price) AS Cheapest_Product_Price FROM Product;
```

- **Result / Output:** Returns 250.00 (the price of the cheapest item in the store).

5. MAX()

- **Definition:** Identifies the maximum (largest) value in a column.
- **Project Example:**

```
SELECT MAX(Price) AS Most_Expensive_Product_Price FROM Product;
```

- **Result / Output:** Returns 65000.00 (the price of the highest-end item).

GROUP BY and HAVING**What is GROUP BY ?**

GROUP BY collapses rows sharing identical values in specified columns into summary groups, allowing aggregate functions to execute per group rather than across the entire table.

- **Project Example:**

```
SELECT c.Category_Name, COUNT(p.Product_ID) AS Number_Of_Products
FROM Category c
LEFT JOIN Product p ON c.Category_ID = p.Category_ID
GROUP BY c.Category_ID, c.Category_Name;
```

What is HAVING ?

HAVING filters the summarized groups **after** aggregate computations take place.

- **Project Example:**

```
SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
FROM Category c
INNER JOIN Product p ON c.Category_ID = p.Category_ID
GROUP BY c.Category_ID, c.Category_Name
HAVING COUNT(p.Product_ID) > 2;
```

Difference: WHERE vs. HAVING

Feature	WHERE Clause	HAVING Clause
Execution Point	Evaluated before rows are grouped (GROUP BY).	Evaluated after groups and aggregates are formed.
Filtering Level	Filters individual rows .	Filters aggregated groups .
Aggregates Allowed?	No. <code>WHERE SUM(Price) > 100</code> throws a syntax error.	Yes. <code>HAVING SUM(Total_Amount) > 5000</code> is valid.

4. SQL Concepts Used in the Project

1. SELECT and Projection

- **Purpose:** Retrieves specified attributes rather than scanning unnecessary columns.
- **Project Example** (05_basic_queries.sql):

```
SELECT Product_Name, Price, Stock_Quantity FROM Product;
```

2. Range Filtering with BETWEEN

- **Purpose:** Inclusive boundary comparison against numeric or date values.
- **Project Example:**

```
SELECT Product_ID, Product_Name, Price FROM Product  
WHERE Price BETWEEN 500.00 AND 3000.00;
```

3. Set Membership with IN

- **Purpose:** Compares an attribute against an explicit set of discrete values.
- **Project Example:**

```
SELECT Product_ID, Product_Name, Category_ID FROM Product  
WHERE Category_ID IN (1, 2);
```

4. Pattern Matching with LIKE and Wildcards

- **Purpose:** Matches substring patterns (% represents zero or more characters).
- **Project Example:**

```
SELECT Supplier_Name, Address FROM Supplier WHERE Address LIKE '%Delhi%';
```

5. Sorting and Limiting (ORDER BY , LIMIT)

- **Purpose:** Orders result sets and extracts the top *N* ranking records.
- **Project Example:**

```
SELECT Product_Name, Stock_Quantity FROM Product  
ORDER BY Stock_Quantity DESC LIMIT 5;
```

6. INNER JOIN

- **Purpose:** Combines records from two or more tables where join conditions match on both sides.
- **Project Example** (06_join_queries.sql):

```
SELECT p.Product_Name, c.Category_Name  
FROM Product p  
INNER JOIN Category c ON p.Category_ID = c.Category_ID;
```

7. LEFT OUTER JOIN

- **Purpose:** Preserves all rows from the left table even when no matching row exists in the right table (unmatched columns return NULL).
- **Project Example (Finding customers who have never placed orders):**

```
SELECT cu.Customer_Name, cu.Phone
FROM Customer cu
LEFT JOIN Sale sa ON cu.Customer_ID = sa.Customer_ID
WHERE sa.Sale_ID IS NULL;
```

8. Scalar Subquery

- **Purpose:** A nested query returning exactly one single value (1×1) used inside a comparison expression.
- **Project Example** (`08_subqueries.sql`):

```
SELECT Product_Name, Price FROM Product
WHERE Price > (SELECT AVG(Price) FROM Product);
```

9. Subquery with `IN` / `NOT IN`

- **Purpose:** Evaluates whether an attribute exists in a dynamically computed subquery list.
- **Project Example (Finding products that have never been sold):**

```
SELECT Product_Name FROM Product
WHERE Product_ID NOT IN (SELECT DISTINCT Product_ID FROM Sale_Details);
```

10. Correlated Subquery

- **Purpose:** An inner query that references attributes of the outer query, executing once for every row evaluated by the outer query.
- **Project Example (Products priced higher than the average of their own category):**

```
SELECT p1.Product_Name, p1.Price, p1.Category_ID
FROM Product p1
WHERE p1.Price > (
    SELECT AVG(p2.Price)
    FROM Product p2
    WHERE p2.Category_ID = p1.Category_ID
);
```

11. `EXISTS` and `NOT EXISTS`

- **Purpose:** Evaluates boolean presence without transmitting data rows, short-circuiting as soon as a single match is detected.
- **Project Example (Suppliers with at least one recorded purchase):**

```
SELECT s.Supplier_Name FROM Supplier s
WHERE EXISTS (
    SELECT 1 FROM Purchase pu WHERE pu.Supplier_ID = s.Supplier_ID
);
```

12. Derived Tables (Subquery in the `FROM` Clause)

- **Purpose:** Treats an aggregated intermediate query as a temporary in-memory table.
- **Project Example:**

```
SELECT ROUND(AVG(Customer_Total_Spend), 2) AS Average_Customer_Spend
FROM (
    SELECT Customer_ID, SUM(Total_Amount) AS Customer_Total_Spend
    FROM Sale GROUP BY Customer_ID
) AS Customer_Spends;
```

5. Database Normalization (1NF, 2NF, 3NF)

Normalization is the systematic process of decomposing database tables to eliminate data redundancy and prevent data modification anomalies:

1. **Insertion Anomaly:** Inability to record certain facts without adding unrelated data (e.g., inability to add a Category without first creating a Product).
2. **Update Anomaly:** Inconsistent data caused by needing to update redundant copies in multiple places (e.g., updating a Supplier's phone number across 500 rows).
3. **Deletion Anomaly:** Loss of unintended data when deleting a record (e.g., deleting a Sale inadvertently deleting the Customer profile).

Raw Unnormalized Data

|

▼ (Remove multi-valued repeating attributes; establish atomic columns and Primary Keys)

First Normal Form (1NF)

|

▼ (Remove Partial Dependencies; all non-key columns depend on entire Composite/Primary Key)

Second Normal Form (2NF)

|

▼ (Remove Transitive Dependencies; non-key columns depend ONLY on Primary Key)

Third Normal Form (3NF) ← Current Project Architecture

Normal Forms Applied to the Schema

1. First Normal Form (1NF)

- **Rules:**

1. All column values must be **atomic** (indivisible single values; no comma-separated lists).
2. Each record must be uniquely identifiable by a **Primary Key**.
3. No repeating column groups (e.g., `Item1`, `Item2`, `Item3`).

- **Compliance in Project:**

- `Sale` does not store `Product_List = "SSD, Keyboard, Mouse"`.
- Multiple items are split into separate atomic rows in `Sale_Details`.
- Every table has a dedicated single-column surrogate primary key (`Category_ID`, `Product_ID`, etc.).

2. Second Normal Form (2NF)

- **Rules:**

1. The table must already be in 1NF.
2. It must have **no partial dependencies** (every non-key attribute must depend on the *entire* primary key, not just a portion of a composite key).

- **Compliance in Project:**

- In `Sale_Details` and `Purchase_Details`, surrogate primary keys (`Sale_Detail_ID`, `Purchase_Detail_ID`) prevent partial dependencies.
- Product details (`Product_Name`, `Price`) are stored in `Product`, not inside `Sale_Details`. `Sale_Details` only stores transaction-specific fields: `Quantity` and `Unit_Price`.

3. Third Normal Form (3NF)

- **Rules:**

1. The table must already be in 2NF.
2. It must have **no transitive dependencies** (non-key attributes must depend *directly* on the primary key, not through another non-key attribute: $X \rightarrow Y$ and $Y \rightarrow Z$ is prohibited).

- **Compliance in Project:**

- `Product` stores `Category_ID` (FK), but does **not** store `Category_Name` or `Category_Description`.
- `Purchase` stores `Supplier_ID` (FK), but does **not** store `Supplier_Name`, `Phone`, or `Address`.

Project-Specific Normalization Viva Scenarios

Scenario 1: Why not store Supplier information inside the `Product` table?

- **Problem:** If `Supplier_Name`, `Supplier_Phone`, and `Supplier_Address` were stored in `Product`:
 1. **Redundancy:** If one supplier provides 50 products, their address is repeated 50 times.
 2. **Update Anomaly:** Changing a supplier's phone requires updating 50 rows; missing one results in conflicting data.
 3. **Insertion Anomaly:** We cannot register a new supplier in the system until they supply at least one product.
 4. **Deletion Anomaly:** Deleting our only product from that supplier erases the supplier's existence from the database.
- **Solution:** Separate `Supplier` table linked to `Purchase` via `Supplier_ID`.

Scenario 2: Why are `Purchase` and `Purchase_Details` separate?

- **Problem:** A purchase order has two levels of data:
 1. Order-level data (applies to the whole invoice: `Purchase_Date`, `Supplier_ID`, `Total_Amount`).
 2. Item-level data (applies to each item: `Product_ID`, `Quantity`, `Unit_Price`).
- **Solution:** Keeping them together would create multi-valued repeating fields (violating 1NF) or force repeating the invoice date and supplier on every row (violating 2NF). Separating them into a parent header (`Purchase`) and child details (`Purchase_Details`) satisfies 3NF.

6. Database Keys

A **Key** is an attribute or set of attributes that uniquely identifies tuples (rows) in a relation and establishes links between tables.

Key Type	Present in Project?	Columns in Project Schema	Explanation
Primary Key (PK)	YES	Category_ID , Product_ID , Supplier_ID , Customer_ID , Purchase_ID , Purchase_Detail_ID , Sale_ID , Sale_Detail_ID	Surrogate integer auto-increment primary keys uniquely identifying every single row across all 8 tables.
Foreign Key (FK)	YES	Product.Category_ID Purchase.Supplier_ID Purchase_Details.Purchase_ID Purchase_Details.Product_ID Sale.Customer_ID Sale_Details.Sale_ID Sale_Details.Product_ID	7 attributes that reference primary keys in parent tables to enforce referential integrity.
Candidate Key	YES	Category.Category_Name	Has a UNIQUE constraint and NOT NULL , making it an alternate candidate key that could uniquely identify a category.
Composite Key	<i>Logical only</i>	(Purchase_ID, Product_ID) in Purchase_Details (Sale_ID, Product_ID) in Sale_Details	Could logically act as a candidate key, but a single-column surrogate key (Sale_Detail_ID) was used for clean indexing and direct reference.

7. Referential Integrity

Referential Integrity is a relational database property guaranteeing that every foreign key value always points to a valid, existing primary key in the referenced parent table.

Parent Table: Customer —[Foreign Key Reference: Referenced PK: Customer_ID]→ Child Table: Sale

- Child Insertion Guard:** If an insert attempts to create a **Sale** with **Customer_ID = 999** and customer **999** does not exist in **Customer** , the InnoDB engine rejects the insert.
- Orphan Prevention:** Orphan records (orders with non-existent customers) cannot be created.
- Parent Deletion Guard (ON DELETE RESTRICT):** If someone executes:

```
DELETE FROM Customer WHERE Customer_ID = 1;
```

MySQL blocks the deletion with an integrity violation error because child rows in **Sale** depend on customer **1** .

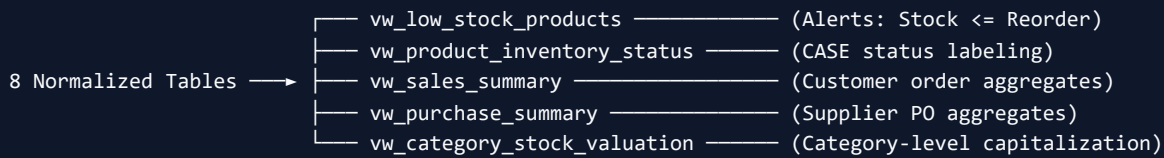
- Parent Deletion Cleanup (ON DELETE CASCADE):** If an order header is cancelled or deleted:

```
DELETE FROM Sale WHERE Sale_ID = 5;
```

The database engine automatically deletes all matching line items in **Sale_Details** , preventing orphaned detail records.

8. Database Views

A **View** is a **virtual table** defined by an underlying SQL **SELECT** query. It does not store physical copies of data; instead, the database executes the view's query dynamically whenever the view is queried.



Detailed Breakdown of the 5 Project Views (09_views.sql)

1. vw_low_stock_products

- **Underlying Tables:** Product p , Category c
- **Purpose:** Identifies products needing urgent replenishment.
- **SQL Mechanism:** INNER JOIN with predicate p.Stock_Quantity <= p.Reorder_Level . Calculates (p.Reorder_Level - p.Stock_Quantity) as Reorder_Deficit .

2. vw_product_inventory_status

- **Underlying Tables:** Product p , Category c
- **Purpose:** Provides a real-time stock status tag for every catalog product.
- **SQL Mechanism:** Uses a conditional CASE statement:

```
CASE
  WHEN p.Stock_Quantity <= p.Reorder_Level THEN 'LOW STOCK ALERT'
  ELSE 'ADEQUATE STOCK'
END AS Stock_Status
```

3. vw_sales_summary

- **Underlying Tables:** Sale sa , Customer cu , Sale_Details sd
- **Purpose:** Summarizes sales invoice metrics including customer contact info, line item counts, and total units sold.
- **SQL Mechanism:** Multi-table join (Sale + Customer + Sale_Details) combined with GROUP BY and aggregates COUNT(sd.Sale_Detail_ID) and COALESCE(SUM(sd.Quantity), 0) .

4. vw_purchase_summary

- **Underlying Tables:** Purchase pu , Supplier s , Purchase_Details pd
- **Purpose:** Summarizes procurement orders per supplier with item counts and total spend.
- **SQL Mechanism:** INNER JOIN and LEFT JOIN grouped by purchase header, computing total units ordered using SUM(pd.Quantity) .

5. vw_category_stock_valuation

- **Underlying Tables:** Category c , Product p
- **Purpose:** Computes inventory capitalization per product category.
- **SQL Mechanism:** LEFT JOIN grouped by category, calculating:

```
COALESCE(SUM(p.Stock_Quantity * p.Price), 0.00) AS Category_Inventory_Valuation
```

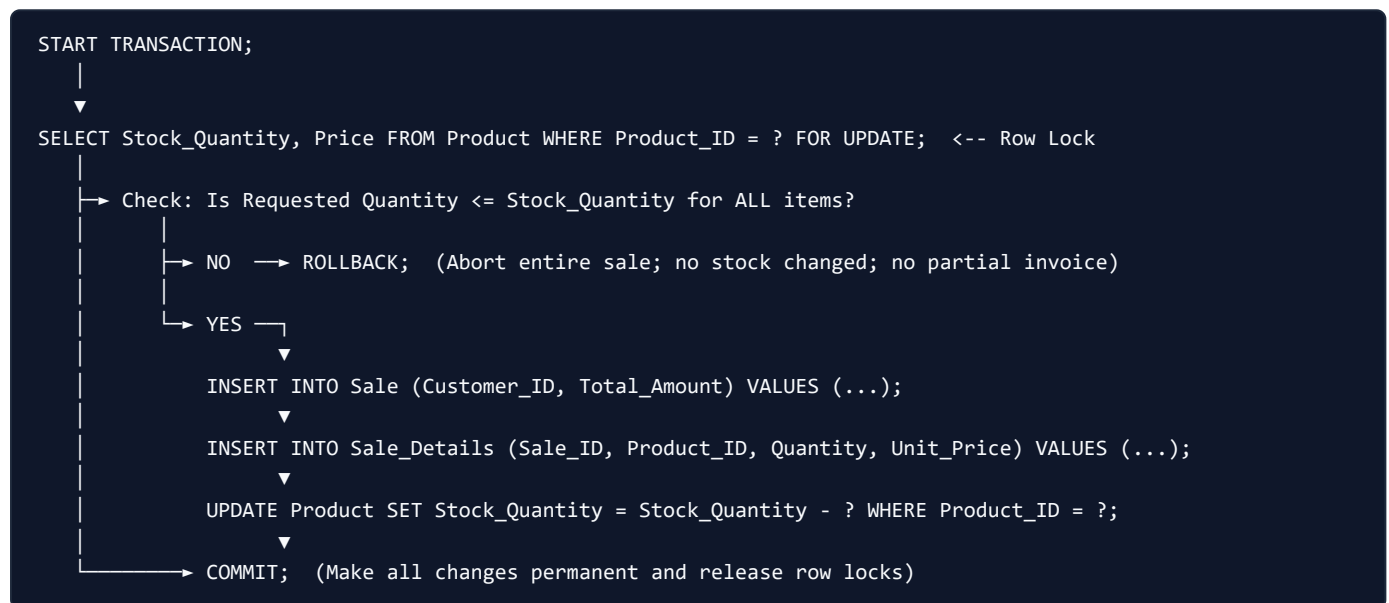
9. Transactions and ACID Properties

A **Transaction** is a logical unit of database work comprising one or more SQL operations that must execute entirely or not at all.

The ACID Properties

Property	DBMS Definition	Implementation in this Project
Atomicity	"All or Nothing." If any statement fails, the entire transaction is rolled back.	When inserting a <code>Sale</code> with 3 items, if stock deduction fails on item 3, all previous inserts and deductions are rolled back.
Consistency	Database moves from one valid state to another , obeying all constraints.	<code>CHECK (Stock_Quantity >= 0)</code> guarantees a sale cannot cause stock to drop below zero.
Isolation	Concurrent transactions execute independently without interference.	Uses <code>SELECT ... FOR UPDATE</code> in InnoDB to place row-level locks on product stock rows during stock checks.
Durability	Once committed, changes are permanent even during system crashes.	Handled by InnoDB's write-ahead Redo Log (<code>ib_logfile</code>), ensuring committed sales persist across power outages.

Step-by-Step Transaction Workflow



Why `ROLLBACK` is Vital:

Without `ROLLBACK`, if a sale had 3 items and the third item ran out of stock:

1. The first two items would be decremented from inventory without payment.
2. An incomplete invoice header would remain orphaned in the database.
3. Financial and inventory accounting would become corrupted.

`ROLLBACK` resets all modified buffers back to their pre-transaction state.

10. Important DBMS Viva Questions & Answers

Q1: What is a DBMS, and why did you choose a Relational DBMS (RDBMS) for this project?

- **VIVA ANSWER:** A DBMS is system software that manages, stores, and retrieves structured data. I chose an RDBMS (MySQL) because an inventory management system requires structured tabular relationships, strict data integrity constraints, and ACID transactions to prevent stock discrepancies.
- **EXPLANATION:** Unlike flat file systems or document stores, an RDBMS enforces schema constraints (Foreign Keys, CHECK constraints) and prevents data anomalies through normalization.

- **PROJECT EXAMPLE:** MySQL InnoDB guarantees that if a customer buys 2 items, inventory drops by 2 and the invoice is created atomically.

Q2: What is the difference between a Primary Key and a Foreign Key?

- **VIVA ANSWER:** A Primary Key uniquely identifies rows within its own table and cannot be null. A Foreign Key is a column in a child table that references the Primary Key of a parent table to enforce referential integrity.
- **EXPLANATION:** Primary Keys maintain Entity Integrity. Foreign Keys maintain Referential Integrity across tables.
- **PROJECT EXAMPLE:** In `Product`, `Product_ID` is the Primary Key. In `Sale_Details`, `Product_ID` is a Foreign Key referencing `Product.Product_ID`.

Q3: What is Referential Integrity, and how does your database maintain it?

- **VIVA ANSWER:** Referential integrity ensures that relationships between tables remain valid, meaning a child record cannot reference a nonexistent parent record. My database enforces it using 7 Foreign Key constraints backed by the InnoDB engine.
- **EXPLANATION:** It prevents orphan rows and restricts parent record deletion when dependent children exist.
- **PROJECT EXAMPLE:** If a user tries to delete a `Customer` who has historical sales orders, MySQL blocks the command with an `ON DELETE RESTRICT` error.

Q4: Why is `Category` separated from `Product` instead of storing the category name in the product row?

- **VIVA ANSWER:** To eliminate data redundancy and satisfy Third Normal Form (3NF). Storing category names inside the product table would repeat category strings across hundreds of items and cause update anomalies.
- **EXPLANATION:** If "Computer Peripherals" was misspelled or renamed, we would have to update hundreds of product rows. With a separate table, we update only one row in `Category`.
- **PROJECT EXAMPLE:** `Category` has `Category_ID` and `Category_Name`. `Product` stores only the 4-byte integer `Category_ID`.

Q5: What is the relationship between `Supplier` and `Purchase` ?

- **VIVA ANSWER:** It is a One-to-Many (1:N) relationship. One supplier can supply multiple purchase orders, but each purchase order is associated with exactly one supplier.
- **EXPLANATION:** The foreign key `Supplier_ID` resides on the `Purchase` table, pointing to `Supplier.Supplier_ID`.
- **PROJECT EXAMPLE:** Supplier `1` supplies Purchase `#101` and Purchase `#105`.

Q6: Why do you need `Purchase_Details` and `Sale_Details` ?

- **VIVA ANSWER:** They are associative detail tables used to resolve Many-to-Many (M:N) relationships between transaction headers (`Purchase` / `Sale`) and catalog items (`Product`).
- **EXPLANATION:** An order can contain multiple items, and a product can appear in multiple orders. Detail tables store line-item intersection data: `Quantity` and `Unit_Price`.

- **PROJECT EXAMPLE:** `Sale_Details` has composite context: `Sale_ID`, `Product_ID`, `Quantity`, and `Unit_Price`.

Q7: What are the CHECK constraints in your project, and why are they needed?

- **VIVA ANSWER:** CHECK constraints enforce domain integrity at the storage engine level to ensure numeric values stay within valid business ranges.
- **EXPLANATION:** They prevent invalid data like negative prices, negative stock, or zero-quantity orders from entering the database.
- **PROJECT EXAMPLE:**
 - `chk_product_price CHECK (Price >= 0)`
 - `chk_product_stock CHECK (Stock_Quantity >= 0)`
 - `chk_sale_detail_qty CHECK (Quantity > 0)`

Q8: What is the difference between `ON DELETE CASCADE` and `ON DELETE RESTRICT`? Where did you use each?

- **VIVA ANSWER:** `RESTRICT` blocks deletion of a parent row if matching children exist. `CASCADE` deletes the parent row and automatically deletes all related child rows.
- **EXPLANATION:** In our business logic, master catalog records (`Category`, `Supplier`, `Customer`, `Product`) must never be accidentally erased if transactions reference them. Conversely, detail line items have no meaning without their parent invoice header.
- **PROJECT EXAMPLE:**
 - `RESTRICT`: Deleting a `Category` with products is blocked.
 - `CASCADE`: Deleting a `Sale` header deletes all its `Sale_Details` rows.

Q9: What is the difference between `WHERE` and `HAVING`?

- **VIVA ANSWER:** `WHERE` filters individual rows **before** aggregation and grouping. `HAVING` filters aggregated groups **after** `GROUP BY` execution.
- **EXPLANATION:** Aggregate functions (like `SUM()` or `COUNT()`) are not permitted in `WHERE` clauses; they must be evaluated using `HAVING`.
- **PROJECT EXAMPLE:**

```
SELECT Category_ID, COUNT(*) FROM Product
WHERE Price > 500 -- Filters individual products
GROUP BY Category_ID
HAVING COUNT(*) > 2; -- Filters grouped categories
```

Q10: What is a Subquery, and how does it differ from a Correlated Subquery?

- **VIVA ANSWER:** A regular subquery executes once independently, passing its result to the outer query. A correlated subquery references columns from the outer query and re-executes for every row processed by the outer query.
- **EXPLANATION:** Correlated subqueries evaluate dynamically per row using outer table values.

- **PROJECT EXAMPLE:**
- *Standard Subquery:* Find products priced above the overall average price (`08_subqueries.sql` , Line 20).
- *Correlated Subquery:* Find products priced above the average price of their own category (`08_subqueries.sql` , Line 71).

Q11: What is `EXISTS` , and why is it often faster than `IN` ?

- **VIVA ANSWER:** `EXISTS` tests for the presence of rows in a subquery and returns boolean true or false. It is often faster than `IN` because it stops scanning as soon as the first matching row is found (short-circuit evaluation).
- **EXPLANATION:** `IN` builds a full intermediate result list in memory before evaluating, whereas `EXISTS` stops at the first match.
- **PROJECT EXAMPLE:** Finding suppliers who have at least one purchase order (`WHERE EXISTS (SELECT 1 FROM Purchase WHERE Supplier_ID = s.Supplier_ID) )`).

Q12: What is a Database View, and why did you use views in your project?

- **VIVA ANSWER:** A View is a virtual table defined by an underlying SQL query that executes dynamically when queried. I used views to encapsulate complex multi-table joins and calculations like low-stock alerts and category inventory valuations.
- **EXPLANATION:** Views simplify reporting queries, reduce query duplication, and decouple data presentation from physical storage.
- **PROJECT EXAMPLE:** `vw_low_stock_products` joins `Product` and `Category` and filters where `Stock_Quantity <= Reorder_Level` .

Q13: What is Normalization, and what normal forms does your database satisfy?

- **VIVA ANSWER:** Normalization is the process of structuring relational tables to reduce data redundancy and eliminate insertion, update, and deletion anomalies. My database satisfies 1NF, 2NF, and 3NF.
- **EXPLANATION:**
- 1NF: Atomic attributes, unique primary keys.
- 2NF: In 1NF with no partial dependencies.
- 3NF: In 2NF with no transitive dependencies.
- **PROJECT EXAMPLE:** Non-key supplier attributes are isolated in `Supplier` , not repeated across `Purchase` or `Product` .

Q14: What is an ACID Transaction, and where is it used in your project?

- **VIVA ANSWER:** ACID stands for Atomicity, Consistency, Isolation, and Durability. It guarantees reliable database processing. I use transactions during Sale and Purchase creation where multiple table writes and stock updates must succeed or fail together.
- **EXPLANATION:** If any step fails during checkout (e.g., negative stock constraint violation), the database issues a `ROLLBACK` to restore previous data.
- **PROJECT EXAMPLE:** Creating a `Sale` , adding `Sale_Details` , and decrementing `Product.Stock_Quantity` are wrapped in a single database transaction using InnoDB row locks (`FOR UPDATE`).

Q15: What happens if two customers buy the last available unit of a product at the exact same second?

- **VIVA ANSWER:** This is a race condition handled by Isolation and Concurrency Control. We use pessimistic row-level locking (`SELECT ... FOR UPDATE`) inside an InnoDB transaction.
- **EXPLANATION:** The first transaction acquires an exclusive row lock on the product row. The second transaction waits. When the first commits, stock drops to 0. The second transaction then reads stock = 0, fails the stock availability check, and triggers a `ROLLBACK`.
- **PROJECT EXAMPLE:** Prevents negative inventory violations (`chk_product_stock`).

Q16: What is the difference between `INNER JOIN` and `LEFT JOIN` ?

- **VIVA ANSWER:** `INNER JOIN` returns only rows that have matching values in both tables. `LEFT JOIN` returns all rows from the left table along with matching rows from the right table (unmatched columns return `NULL`).
- **EXPLANATION:** An `INNER JOIN` excludes unmatched entities; a `LEFT JOIN` preserves all left-side rows.
- **PROJECT EXAMPLE:**
- `INNER JOIN` : Lists products with their category names (excludes products without a category, if any).
- `LEFT JOIN` : Lists all categories, including categories that currently contain 0 products.

Q17: What is the purpose of the `COALESCE()` function in your views?

- **VIVA ANSWER:** `COALESCE()` returns the first non-null expression in its argument list. It is used to convert `NULL` values resulting from outer joins into `0` or `0.00`.
- **EXPLANATION:** If a product has never been sold, a `LEFT JOIN` on `Sale_Details` results in `SUM(Quantity)` evaluating to `NULL`. `COALESCE(SUM(Quantity), 0)` turns it into `0`.
- **PROJECT EXAMPLE:** Used in `vw_category_stock_valuation` and `vw_sales_summary` to prevent calculations from returning `NULL`.

Q18: What is a Composite Key, and why did you choose surrogate keys instead?

- **VIVA ANSWER:** A Composite Key is a primary key composed of two or more columns (e.g., `Sale_ID` + `Product_ID`). I used single-column surrogate keys (`Sale_Detail_ID`) for simpler foreign key references, cleaner indexing, and better join performance.
- **EXPLANATION:** While (`Sale_ID`, `Product_ID`) is a candidate key, surrogate keys provide a stable, unchanging integer identifier for each row.
- **PROJECT EXAMPLE:** `Sale_Details` has `Sale_Detail_ID INT AUTO_INCREMENT PRIMARY KEY`.

11. TOP 25 DBMS Viva Points to Memorize

1. **Database Engine:** MySQL with **InnoDB** storage engine (chosen for foreign keys, row-level locking, and ACID transactions).
2. **Schema Size:** Exactly **8 relational tables** and **5 analytical views**.
3. **Core Tables:** `Category`, `Product`, `Supplier`, `Customer`, `Purchase`, `Purchase_Details`, `Sale`, `Sale_Details`.

4. **Primary Keys:** Every table uses an auto-increment surrogate primary key (`*_ID`).
5. **Foreign Keys:** Exactly **7 Foreign Key constraints** maintain referential integrity across the schema.
6. **M:N Resolution:** Many-to-Many relationships (`Purchase - Product` and `Sale - Product`) are resolved using two bridge tables: `Purchase_Details` and `Sale_Details` .
7. **Entity Integrity:** Enforced by Primary Key constraints; no primary key can be `NULL` or duplicated.
8. **Referential Integrity:** Enforced by Foreign Keys; child records cannot point to non-existent parents.
9. **Domain Integrity:** Enforced using `CHECK` constraints (e.g., `Price >= 0` , `Stock_Quantity >= 0` , `Quantity > 0`).
10. **Unique Constraint:** `Category_Name` has a `UNIQUE` constraint (`uq_category_name`) preventing duplicate category labels.
11. **Deletion Safety:** Parent master tables use `ON DELETE RESTRICT` to prevent deleting customers, products, or suppliers with transaction histories.
12. **Line Item Cleanup:** Detail tables use `ON DELETE CASCADE` so deleting a parent invoice header purges its child line items.
13. **Update Propagation:** All foreign keys use `ON UPDATE CASCADE` so primary key updates reflect across child records.
14. **Normalization Level:** The schema is fully normalized up to **Third Normal Form (3NF)**.
15. **1NF Compliance:** All attributes are atomic; no repeating column groups exist.
16. **2NF Compliance:** In 1NF with zero partial functional dependencies on primary keys.
17. **3NF Compliance:** In 2NF with zero transitive dependencies (non-key columns depend only on the primary key).
18. **Aggregates Used:** `COUNT()` , `SUM()` , `AVG()` , `MIN()` , `MAX()` .
19. **WHERE vs. HAVING :** `WHERE` filters rows before grouping; `HAVING` filters aggregated groups after `GROUP BY` .
20. **Views Implemented:** 5 views (`vw_low_stock_products` , `vw_product_inventory_status` , `vw_sales_summary` , `vw_purchase_summary` , `vw_category_stock_valuation`).
21. **View Nature:** Virtual tables storing SQL queries, not physical duplicate tables.
22. **Correlated Subqueries:** Demonstrated in `08_subqueries.sql` by finding products priced above their own category average.
23. **ACID Properties:** Atomicity (all-or-nothing), Consistency (rules maintained), Isolation (concurrent safety), Durability (crash resilience).
24. **Concurrency Control:** Pessimistic row-level locking via `SELECT ... FOR UPDATE` prevents inventory overselling during simultaneous purchases.
25. **Rollback Purpose:** Aborts a transaction on validation failure or constraint error, restoring table data to its original state.